

Nama: Carlos Fadellilah Rama

Kelas: TI-2E

Matkul: Praktikum PBO

Jobsheet 03 – Enkapsulasi PBO Polinema

Materi: Enkapsulasi, Access Modifier, Getter Setter, Konstruktor.

Percobaan 1 — Enkapsulasi

Tujuan: Membuat class Motor yang masih belum menggunakan enkapsulasi sehingga atribut dapat diakses langsung.

Jobsheet 03 - Enkapsulasi PBO.docx

1. Class Motor.java

```
public class Motor {  
  
    public int kecepatan = 0;  
    public boolean kontakOn = false;  
  
    public void printStatus() {  
        if (kontakOn) {  
            System.out.println("Kontak On");  
        } else {  
            System.out.println("Kontak Off");  
        }  
  
        System.out.println("Kecepatan : " + kecepatan + "\n");  
    }  
}
```

2. Class MotorDemo.java

```
public class MotorDemo {  
  
    public static void main(String[] args) {  
  
        Motor motor = new Motor();  
  
        motor.printStatus();  
  
        motor.kecepatan = 50;  
        motor.printStatus();  
    }  
}
```

```
}  
}
```

3. Output

```
PS C:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO prak> cd "c:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO prak\job 3\motorenkapsulasi\" ; if ($?) { javac MotorDemo.java } ; if ($?) { java MotorDemo  
Kontak OFF  
Kecepatan 0  
  
Kontak OFF  
Kecepatan 50
```

Analisis

Program masih memiliki kelemahan karena atribut kecepatan dapat diubah langsung walaupun kontak motor masih OFF. Hal ini menunjukkan belum adanya proses enkapsulasi.

Percobaan 2 — Access Modifier (Enkapsulasi yang Benar)

Tujuan: Menggunakan private agar atribut hanya bisa diakses melalui method.

1. Class Motor.java

```
public class Motor {  
  
    private int kecepatan = 0;  
    private boolean kontakOn = false;  
  
    public void nyalakanMesin() {  
        kontakOn = true;  
    }  
  
    public void matikanMesin() {  
        kontakOn = false;  
        kecepatan = 0;  
    }  
  
    public void tambahKecepatan() {  
        if (kontakOn == true) {  
            kecepatan += 5;  
        } else {  
            System.out.println("Kecepatan tidak bisa bertambah karena Mesin Off!");  
        }  
    }  
  
    public void kurangiKecepatan() {
```

```

        if (kontakOn == true) {
            kecepatan -= 5;
        } else {
            System.out.println("Kecepatan tidak bisa berkurang karena Mesin Off!");
        }
    }

    public void printStatus() {
        if (kontakOn == true) {
            System.out.println("Kontak On");
        } else {
            System.out.println("Kontak Off");
        }

        System.out.println("Kecepatan : " + kecepatan + "\n");
    }
}

```

2. Class MotorDemo.java

```

public class MotorDemo {

    public static void main(String[] args) {

        Motor motor = new Motor();

        motor.printStatus();

        motor.tambahKecepatan();

        motor.nyalakanMesin();

        for (int i = 0; i < 5; i++) {
            motor.tambahKecepatan();
        }

        motor.printStatus();

        motor.kurangiKecepatan();
        motor.printStatus();

        motor.matikanMesin();
        motor.printStatus();
    }
}

```

}

3. Output

```
PS C:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO prak\job 3\motorenkapsulasi> cd "c:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO prak\job 3\motorenkapsulasi\" ; if ($?) { javac MotorDemo.java } ; if ($?) { java MotorDemo }
Kontak Off
Kecepatan : 0

Kecepatan tidak bisa bertambah karena Mesin Off!
Kontak On
Kecepatan : 25

Kontak On
Kecepatan : 20

Kontak Off
Kecepatan : 0
```

Jawaban Pertanyaan Percobaan 2

1. Mengapa muncul peringatan "Kecepatan tidak bisa bertambah karena Mesin Off!"?

Karena method `tambahKecepatan()` memeriksa kondisi `kontakOn`. Jika nilainya `false`, penambahan kecepatan diblokir.

2. Mengapa atribut kecepatan dan kontakOn dibuat private?

Agar atribut tidak dapat diakses atau diubah secara langsung dari luar class sehingga data lebih aman dan terkontrol.

3. Ubah class Motor sehingga kecepatan maksimal 100.

```
public void tambahKecepatan() {
    if (kontakOn) {
        if (kecepatan < 100) {
            kecepatan += 5;
        } else {
            System.out.println("Kecepatan sudah maksimal!");
        }
    } else {
        System.out.println("Kecepatan tidak bisa bertambah karena Mesin Off!");
    }
}
```

Percobaan 3 — Getter dan Setter

Tujuan: Menggunakan getter dan setter pada class Anggota.

1. Class Anggota.java

```
public class Anggota {  
  
    private String nama;  
    private String alamat;  
    private int simpanan;  
  
    public void setNama(String nama) {  
        this.nama = nama;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public void setAlamat(String alamat) {  
        this.alamat = alamat;  
    }  
  
    public String getAlamat() {  
        return alamat;  
    }  
  
    public int getSimpanan() {  
        return simpanan;  
    }  
  
    public void setor(int uang) {  
        simpanan += uang;  
    }  
  
    public void pinjam(int uang) {  
        simpanan -= uang;  
    }  
}
```

2. Class KoperasiDemo.java

```
public class KoperasiDemo {
```

```

public static void main(String[] args) {

    Anggota donny = new Anggota();

    donny.setNama("Donny");
    donny.setAlamat("Malang");

    System.out.println("Nama    : " + donny.getNama());
    System.out.println("Alamat  : " + donny.getAlamat());
    System.out.println("Simpanan : Rp " + donny.getSimpanan());

    donny.setor(100000);

    System.out.println("\nSetor 100000");
    System.out.println("Simpanan : Rp " + donny.getSimpanan());

    donny.pinjam(5000);

    System.out.println("\nPinjam 5000");
    System.out.println("Simpanan : Rp " + donny.getSimpanan());
}
}

```

3. Output

```

Nama      : Donny
Alamat    : Malang
Simpanan  : Rp 0

Setor 100000
Simpanan  : Rp 100000

Pinjam 5000
Simpanan  : Rp 95000

```

Percobaan 4 — Konstruktor dan Instansiasi

Tujuan: Menggunakan konstruktor agar objek langsung memiliki nilai awal.

1. Rubah Class Koperasi Demo

```

public class KoperasiDemo {

    public static void main(String[] args) {

        Anggota anggota1 = new Anggota();
    }
}

```

```

        anggota1.setNama("Iwan Setiawan");
        anggota1.setAlamat("Jalan Sukarno Hatta no 10");
        anggota1.setor(100000);
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " +
        anggota1.getSimpanan());

        anggota1.pinjam(5000);
        System.out.println("Simpanan " + anggota1.getNama()+ " : Rp " +
        anggota1.getSimpanan());
    }
}

```

Hasil Output

```

Simpanan Iwan Setiawan : Rp 100000
Simpanan Iwan Setiawan : Rp 95000

```

2. Rubah Class Anggota

```

public class Anggota {

    private String nama;
    private String alamat;
    private int simpanan;

    Anggota(String nama, String alamat) {
        this.nama = nama;
        this.alamat = alamat;
        this.simpanan = 0;
    }

    public String getNama() {
        return nama;
    }

    public String getAlamat() {
        return alamat;
    }

    public int getSimpanan() {
        return simpanan;
    }
}

```

```

    public void setor(int uang) {
        simpanan += uang;
    }

    public void pinjam(int uang) {
        simpanan -= uang;
    }
}

```

3. Output

```

Nama Anggota : Iwan : Rp 0
Simpanan Iwan Setiawan : Rp 100000
Simpanan Iwan Setiawan : Rp 95000

```

Jawaban Pertanyaan Percobaan 3 dan 4

Jawaban berikut mengikuti materi jobsheet.

No Pertanyaan

- 1 Getter adalah method untuk mengambil nilai atribut private, sedangkan setter adalah method untuk mengubah nilai atribut private.
- 2 getSimpanan() digunakan untuk melihat atau mengambil nilai simpanan tanpa mengubah nilainya.
- 3 Method setor() digunakan untuk menambah saldo atau simpanan anggota.
- 4 Konstruktor adalah method khusus yang otomatis dijalankan ketika objek dibuat menggunakan new dan tidak memiliki tipe return.
- 5 Aturan konstruktor: (1) nama sama dengan class, (2) tidak memiliki tipe return, (3) tidak boleh menggunakan abstract, static, final, atau synchronized.
- 6 Boleh. Konstruktor dapat bertipe private, biasanya digunakan pada Singleton atau Utility Class.

- 7 Passing parameter digunakan ketika objek membutuhkan nilai awal tertentu saat dibuat, misalnya nama dan alamat anggota.
- 8 Inisialisasi atribut adalah memberi nilai awal pada atribut, sedangkan instansiasi atribut adalah membuat objek dari suatu class menggunakan new.
- 9 Inisialisasi method adalah mendefinisikan atau membuat method di dalam class, sedangkan instansiasi method adalah memanggil method melalui objek yang telah dibuat.

Tugas

1. Tugas 1

```
Name : James  
Age  : 30
```

Hasil Output Program

2. Tugas 2

Nilai umur menjadi 30 karena method `setAge()` menerapkan validasi yang membatasi umur maksimal 30 tahun. Ini merupakan contoh enkapsulasi, yaitu data tidak dapat diubah sembarangan dari luar class, melainkan harus melalui method yang mengatur aturan perubahan data.

3. Tugas 3

Pada method `setAge()` ditambahkan dua proses validasi:

- Jika umur kurang dari 18, maka nilai age otomatis menjadi 18.
- Jika umur lebih dari 30, maka nilai age otomatis menjadi 30.
- Jika umur berada di antara 18 sampai 30, maka nilai disimpan sesuai input pengguna.

Dengan cara ini, atribut age tetap terlindungi melalui enkapsulasi, karena perubahan nilai hanya bisa dilakukan melalui method `setAge()` yang memiliki aturan validasi.

```
Umur setelah input 15 : 18  
Umur setelah input 22 : 22  
Umur setelah input 35 : 30
```

4. Tugas 4

```
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muat/menurunkan barang seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muat/menurunkan barang seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg
```

Pada program ini diterapkan enkapsulasi karena seluruh atribut pada class Kontainer dibuat private. Penambahan dan pengurangan muatan tidak dilakukan secara langsung, tetapi melalui method tambahMuatan() dan turunkanMuatan(). Method tambahMuatan() memeriksa apakah total muatan melebihi kapasitas maksimal 5000 kg, sedangkan turunkanMuatan() memastikan muatan yang dibongkar tidak melebihi muatan yang tersedia. Hasil output menunjukkan validasi tersebut bekerja sesuai soal pada Jobsheet Polinema.

5. Tugas 5

```
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muat/menurunkan barang seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muat/menurunkan barang seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg
```

Pada Tugas 5, method turunkanMuatan() dimodifikasi dengan menambahkan aturan bahwa berat barang yang dibongkar dalam satu kali proses tidak boleh melebihi 50% dari berat muatan saat ini. Program menghitung batas maksimal menggunakan:

```
double batasMaksimal = beratMuatanSaatIni * 0.5;
```

Jika berat yang dimasukkan lebih besar dari batasMaksimal, sistem menolak pembongkaran dan menampilkan pesan peringatan sesuai ketentuan jobsheet. Jika tidak melebihi batas tersebut, muatan berhasil dikurangi dari beratMuatanSaatIni.

6. Tugas 6

```
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Masukkan berat muatan yang ingin ditambahkan (kg): 2344
Berat muatan saat ini: 2344.0 kg

Masukkan berat muatan yang ingin dibongkar (kg): 4566
Maaf, demi keselamatan, pembongkaran muatan satu kali jalan tidak boleh melebihi 50% dari muatan saat ini!
Berat muatan saat ini: 2344.0 kg
```

Pada Tugas 6, class TestLogistik dimodifikasi dengan menambahkan Scanner dari package java.util sehingga pengguna dapat memasukkan berat muatan melalui keyboard. Nilai yang diinput disimpan ke variabel tambah dan turun, kemudian dikirim sebagai parameter ke method tambahMuatan() dan turunkanMuatan(). Dengan cara ini, program menjadi interaktif karena data tidak lagi ditulis langsung di dalam kode, melainkan dimasukkan saat program dijalankan melalui terminal.

7. Tugas 7

```
=== DATA TIKET 1 ===
Judul Film      : Avengers: Endgame
Harga Dasar     : Rp 50000.0
Status Pembayaran : false

Setelah melakukan pembayaran
Status Pembayaran : true

=== DATA TIKET 2 ===
Judul Film      : Naruto The Movie
Harga Dasar     : Rp 35000.0
Status Pembayaran : false
```

Class Tiket menerapkan konsep enkapsulasi dengan menjadikan atribut judulFilm, hargaDasar, dan statusPembayaran sebagai private, sehingga tidak dapat diakses langsung dari luar class. Nilai hargaDasar divalidasi di dalam konstruktor; jika pengguna memasukkan nilai negatif, sistem otomatis menggunakan harga default Rp35.000 sesuai ketentuan soal. Status pembayaran diatur false saat objek dibuat dan hanya dapat diubah melalui method lakukanPembayaran(). Atribut statusPembayaran hanya memiliki getter sehingga bersifat read-only dari luar class.